

Session 4 Homework — Recently Viewed, Not Saved Yet

CORE, STRETCH, and CHALLENGE are all required — every one of them must be completed. The three names describe increasing difficulty only, nothing about which parts you submit.

Context

This extends the same B4F Hub project. It is **not** a separate project — it continues from where Session 4 ended, persistence included: `savedOpportunities.savedIds` already survives a reload.

In class you built one selector that reads both slices at once (`selectTotalActivityCount`), one control that dispatches to both slices from a single handler (Reset activity), and — after the teamwork — real, working `localStorage` persistence for exactly one piece of state (`savedOpportunities.savedIds`). This homework reuses all three of those ideas, one tier at a time.

There is no new slice anywhere in this homework. Both `savedOpportunities` and `recentlyViewed` already hold everything needed.

The product story

Product has a new request: right now, only *saved* opportunities survive a reload — anyone who closes the tab loses their recently-viewed list. Product wants recently-viewed opportunities to survive a reload too, but only once someone has actually acted on the "not saved yet" list this homework builds.

This is a **new, deliberate decision about one more piece of state** — the same kind of decision that made `savedIds` persist in class and nothing else. It does not mean every future slice should persist by default; it means someone looked at `recentlyViewed` specifically and decided it should.

Goal

Answer the question: *which recently-viewed opportunities have never been saved?* — make that answer a real, visible, reusable part of the app, coordinate an existing action across all of them at once, and then persist `recentlyViewed` itself, the way `savedIds` was persisted in class.

What to build

A new function in `client/src/store/selectors.ts`, the UI built on it, a "Save all" control, and a second, independent manual `localStorage` persistence path for `recentlyViewed.entries`.

CORE (required) — which recently-viewed entries aren't saved?

1. Add `selectRecentlyViewedNotSaved(state)` to `client/src/store/selectors.ts`, alongside `selectTotalActivityCount` and `selectHasAnyActivity`. It must return the list of recently-viewed entries whose `id` does not appear in the saved ids — reuse each slice's own named selectors to read

them (`selectSavedIds` , `selectRecentlyViewed`); do not reach into either slice's raw state shape by hand.

2. In `RecentlyViewedList` , read this new selector and show a small note whenever the list it returns is non-empty, naming how many recently-viewed entries are not yet saved (for example: "2 of these aren't saved yet.").
3. The note must disappear once nothing in the recently-viewed strip is unsaved, and must never show a count of 0.

When CORE is done, the note appears and disappears correctly, and updates the moment an opportunity is saved or a new one is viewed — no page reload needed.

STRETCH (required) — Save all, coordinating actions you already have

1. Add a "Save all" control next to the CORE note (only visible when the not-yet-saved list has at least one entry) that saves every currently not-yet-saved, recently-viewed opportunity in one click.
2. Do this by dispatching the app's existing `toggleSaved` action **once per entry**, using `.forEach()` :

TS

```
notSaved.forEach((entry) => dispatch(toggleSaved(entry.id)));
```

Do not write `dispatch(notSaved.map((entry) => toggleSaved(entry.id)))` . `dispatch` takes one action object at a time — it has no idea what to do with an array of actions. `.forEach()` calls `dispatch` separately for each entry, which is what actually sends each action through the store, one at a time; `.map()` alone never calls `dispatch` at all except the one time on the outer array.

1. After clicking it, every opportunity that was in the not-yet-saved list must now show as saved everywhere that state is shown (the star on its card, the Navbar badge, the Activity summary), and the CORE note must disappear (nothing left to report).

Prove it needs zero new persistence code. "Save all" only dispatches `toggleSaved` — an action that already exists and that the class's `store.subscribe` callback already reacts to. Reload the page after clicking "Save all": everything it saved is still saved. Write down, in your own words, why this works even though you didn't add a single line of persistence code for it. The chain is:

TEXT

```
Save all → dispatch(toggleSaved(id)) for each entry → savedOpportunities.savedIds changes
→ the EXISTING store.subscribe callback runs (as it does after every dispatch)
→ savedIds is written to localStorage
→ reload → savedIds is read back → the same opportunities show as saved
```

This is Test D below — do not skip proving it.

CHALLENGE (required) — persist Recently Viewed itself

Now build the persistence Product actually asked for: make `recentlyViewed.entries` survive a reload too, following the same manual pattern taught in class for `savedIds` — but as its own, independent path, not a reuse of the saved-opportunities key.

1. In `client/src/store/recentlyViewedSlice.ts`, add a new exported storage-key constant with its own name and its own value — for example `RECENTLY_VIEWED_ENTRIES_KEY = "recentlyViewedEntries"`. **This must be a different string from `SAVED_OPPORTUNITY_IDS_KEY`.** Reusing that key, or hard coding its string, is not allowed — the two pieces of state must not overwrite each other in `localStorage`.
2. Add a loader function that mirrors the one from class exactly in shape: read the key, and if nothing is stored or `JSON.parse` throws, fall back to `[]`. No schema validation, no checking that each entry has the right fields — the same honest fallback as `loadSavedIds`, nothing more.
3. Use that loader as the slice's `initialState.entries` — the same way `loadSavedIds()` is used in `savedOpportunitiesSlice.ts`.
4. **Extend** the existing `store.subscribe` callback in `client/src/store/store.ts` to also write `recentlyViewed.entries` to the new key. **Do not add a second `store.subscribe(...)` call.** There is exactly one subscription; it now writes to two keys inside the same callback.
5. Every existing recently-viewed behaviour must keep working exactly as before: newest-first ordering, de-duplication when re-visiting the same opportunity, the 5-entry cap, removing one entry, clearing all of them, and excluding the opportunity currently open on its own detail page. None of that logic changes — only where `initialState.entries` comes from, and what `store.subscribe` now also writes.

Rules

Allowed — everything taught by the end of Session 4, persistence block included: every hook and selector pattern from CORE and the teamwork (`useAppDispatch`, `useAppSelector`, composed selectors, a selector calling another selector, one handler dispatching more than one action), the manual `localStorage` persistence pattern from LC-14...19 (a storage-key constant, a loader with a try/catch-to-`[]` fallback used in `initialState`, a `store.subscribe` callback), and ordinary array methods (`.filter()`, `.forEach()`, `.includes()`).

Not yet (do not use these) — Redux Persist (the library), any custom middleware, any thunk (`createAsyncThunk` included), `extraReducers`, RTK Query, `createSelector` (reselect), `createEntityAdapter`, `useReducer`, a backend/database change, IndexedDB, a new `sessionStorage` implementation, real authentication, or any schema-validation/migration/versioning library for the stored data. If you find yourself reaching for one of these, you have left the scope of this homework.

Do not change the server. Do not change `NotificationContext`. Do not add a third slice. Do not touch `savedOpportunitiesSlice.ts` — its persistence code from class already works and needs no changes here.

Terminology. Say "we are persisting data used by Redux" or "we're adding persistence around Redux," the way class did. Never say "Redux now persists the state" — `configureStore` is not touched by this homework either; every persisted key is still ordinary application code reading and writing `localStorage`.

Test it

A — saved opportunities still persist. Save an opportunity, reload: it is still saved. (This already worked at the end of class — confirm your CHALLENGE changes didn't break it.)

B — recently viewed now persists. View two or three opportunities, reload: the same recently-viewed list is still there, in the same order.

C — remove and clear persist too. Remove one recently-viewed entry, reload: it stays removed. Clear the whole recently-viewed list, reload: it stays empty.

D — Save all persists through the existing path. With at least two unsaved, recently-viewed entries showing, click "Save all," then reload: every one of them is still saved, and the not-saved note is still gone. You did not write any new persistence code to make this true — see STRETCH's diagram above and be ready to explain why it works.

E — write it down. In 3 to 5 sentences, in your own words, explain: why Redux alone (without this homework's code) would not have made `recentlyViewed` survive a reload; what `store.subscribe` actually contributed to making it survive; what "rehydration" means for the `entries` array specifically; and why only `savedIds` and `recentlyViewed.entries` persist while `NotificationContext` and any loading/error state still do not. Put this at the top of your PR description or in a short `NOTES.md` next to your changes — a working reviewer needs to be able to find it without hunting.

Also run: `npm run build` and `npm run lint`, both with 0 errors.

What correctly still does not persist

`NotificationContext` and any other transient UI state (like `savedOnly`, or loading/error flags) still reset on every reload, on purpose — this homework never touches them, and persisting everything by default was never the point. Persistence stays a deliberate, one-piece-of-state-at-a-time decision, made explicitly for `savedIds` in class and for `recentlyViewed.entries` here.

Done when

- CORE, STRETCH and CHALLENGE are all implemented — all three are required.
- `selectRecentlyViewedNotSaved` exists exactly once and is called from every component that needs it.
- "Save all" dispatches `toggleSaved` once per entry via `.forEach()` — never `dispatch(array.map(...))`.
- `recentlyViewed.entries` persists under its own key, distinct from `SAVED_OPPORTUNITY_IDS_KEY`, written by the one, extended `store.subscribe` callback — not a second subscription.
- All of Tests A–E pass, including the written explanation for Test E.
- `savedOpportunitiesSlice.ts` is unchanged.
- `npm run build` and `npm run lint` pass with 0 errors.

Working time

Expect about **2 to 3 hours** — CORE and STRETCH extend Session 4's own patterns directly; CHALLENGE is the part that takes real care, since it touches the shared `store.subscribe` callback and must not break the persistence that already works.

Submitting

Commit and push to your own `student-<your-github-username>` branch, as usual. Do not push to `main`.